MapReduce Theory

Think of **MapReduce** as a system that helps process **very large files** by using **many computers together**.

Instead of one computer doing all the work slowly, Hadoop splits the file and many machines work at the same time.

One-Line Definition

MapReduce = Divide big data work into small tasks, process in parallel, combine final result.

Why We Need It

Suppose you have a **1 GB file** (or much larger).

If one computer reads and processes everything:

- Slow
- Heavy memory usage
- Risk of failure

MapReduce solves this by distributing work.

Two Main Phases

Phase	Meaning	Work
Map	Break & transform	Reads chunks and creates key-value output
Reduce	Combine	Merges same keys and gives final result

Very Simple Real Example

Suppose file contains words:

apple mango apple orange mango apple

We want count of each word.

Final output:

apple 3

mango 2

orange 1

MapReduce does this automatically.

Now Your Question: 1 GB File — How It Works

Step 1: File Stored in HDFS

Your **1 GB file** is stored in Hadoop Distributed File System (HDFS).

HDFS splits file into blocks.

Let's assume **128 MB block size**.

Number of blocks:

$$\frac{1024\ \text{MB}}{128\ \text{MB}}=8$$

So **1 GB file = 8 blocks**

Blocks:

B1 B2 B3 B4 B5 B6 B7 B8

These blocks are stored across multiple machines (DataNodes).

Step 2: Mapper Starts

Usually **one mapper per block**.

So:

- Mapper1 → Block1
- Mapper2 → Block2
- Mapper3 → Block3
- ...
- Mapper8 → Block8

If cluster has 4 machines:

- 4 mappers run first in parallel
- next 4 run after slots free

So work is faster.

Important Concept: Data Locality

Hadoop sends **code to machine where data exists**.

Not data to code.

Why?

- Code is small

- Data is huge

This saves network time.

Example: Word Count on 1 GB File

Suppose Block1 contains:

apple mango apple

Mapper output:

(apple,1)

(mango,1)

(apple,1)

Block2 contains:

mango orange

Mapper output:

(mango,1)

(orange,1)

All blocks do same.

Step 3: Shuffle and Sort

Hadoop groups same keys together:

apple -> 1,1,1,...

mango -> 1,1,...

orange -> 1,...

This is automatic.

Step 4: Reducer

Reducer adds values:

apple -> 3

mango -> 2

orange -> 1

Final output stored in HDFS.

Full Flow of 1 GB File

1 GB File

↓

Split into 8 blocks

↓

8 Mappers process blocks

↓

Intermediate (word,1)

↓

Shuffle + Sort

↓

Reducer adds counts

↓

Final Result

Another Example: Sum Sales Data

If 1 GB file contains:

Hyderabad,500

Bangalore,300

Hyderabad,200

Map:

(Hyderabad,500)

(Bangalore,300)

(Hyderabad,200)

Reduce:

Hyderabad 700

Bangalore 300

Why MapReduce Is Powerful

Without MapReduce

1 computer processes 1 GB alone.

With MapReduce

Many machines process chunks together.

Result:

- Faster
- Scalable
- Fault tolerant

If One Machine Fails?

Hadoop re-runs task on another machine because data has replicas.

That's a big advantage.

Interview Definitions

Mapper

Processes input split and gives intermediate key-value pairs.

Reducer

Combines values of same key and gives final output.

Input Split

Logical chunk of file for mapper.

Shuffle

Moves mapper output to reducers.

Very Easy Memory Trick

MAP = Make small pieces

REDUCE = Return final combined answer

Honest Industry Truth

MapReduce is older tech. Today companies often use:

- Apache Spark
- Databricks

But MapReduce theory is still very important because it teaches distributed processing basics.

What Happens on 1 GB File in One Sentence

Hadoop breaks 1 GB into 8 blocks, runs multiple mappers in parallel, groups outputs, reducers combine them, and stores final result.

MapReduce

First Understand Big Picture

Hadoop has 2 main parts:

Part	Work
HDFS	Stores huge files
MapReduce	Processes huge files

So:

- **HDFS = Storage**
- **MapReduce = Processing Engine**

Why MapReduce Came?

Normal programs work well when data is in **one computer**.

But if data is:

- 500 GB
- 1 TB
- 10 TB

One machine becomes slow.

So Hadoop uses:

- many machines
- split file into blocks
- process blocks in parallel

This is MapReduce.

Core Idea in One Line

Break big work into small tasks, run on many machines, combine final result.

Two Phases of MapReduce

1. MAP Phase

Reads input data and converts into key-value pairs.

2. REDUCE Phase

Collects same keys and combines results.

Example From Your Notes: Word Count

Suppose file contains:

hello my name is sumit

i love to teach big data

hello this is me

We want output:

hello = 2

is = 2

sumit = 1

data = 1

Step 1: File Split Into Blocks

Suppose 500 MB file.

Default Hadoop block size (older notes): **128 MB**

So:

500 MB / 128 MB ≈ 4 blocks

So Hadoop makes:

- Block1
- Block2
- Block3
- Block4

Each block stored in different DataNodes.

Let's say that user has written map-reduce code

the code will be transferred to each datanode

same code will be run on each datanode where your data is stored as blocks

these 4 nodes work in parallel here but works on different blocks of data at a time.

Step 2: Mapper Runs on Each Block

Each block gets one Mapper.

So:

- Mapper1 works on Block1
- Mapper2 works on Block2
- Mapper3 works on Block3
- Mapper4 works on Block4

This gives **parallel processing**.

Mapper Output

Mapper converts each word into:

(word,1)

Example:

Input line:

hello my name is sumit

Mapper output:

(hello,1)

(my,1)

(name,1)

(is,1)

(sumit,1)

the output of the mapper can go to any one of these machines or it can go to a different machine and reduce activity will takes place.

since 4 mappers are there ,so we get 4 results and all are sent to a datanode for reduce activity

---

Why (word,1)?

Because later reducer adds them.

Step 3: Shuffle + Sort

All mapper outputs sent together.

Same keys grouped.

Like:

hello -> (1,1)

is -> (1,1)

sumit -> (1)

This stage is automatic Hadoop work.

Called:

Shuffle & Sort

Very important.

Step 4: Reducer

Reducer adds values.

hello -> 1+1 = 2

is -> 1+1 = 2

sumit -> 1

Final output:

(hello,2)

(is,2)

(sumit,1)

Why It Is Powerful

Instead of 1 machine reading full 500MB:

4 machines read 4 blocks together.

So faster.

Important Line in Your Notes: Data Locality

Your image says:

Is data going to code or code going to data?

Answer:

Hadoop sends code to where data is stored.

Why?

Because:

- Data is huge
- Code is small

So better move code.

This is called:

Data Locality

Very important interview topic.

Example

Suppose Block1 stored in Machine A.

Then Mapper code runs on Machine A.

No need to move 128MB data across network.

Fast.

If 1 GB File?

Your notes say:

1 GB = 8 blocks

Then:

- 8 mappers created
- maybe 4 machines run 4 first
- next 4 after resources free

Real Internal Flow

Big File in HDFS

↓

Split into blocks

↓

Many Mappers

↓

(word,1)

↓

Shuffle + Sort

↓

Reducer

↓

(word,total_count)

One Line Definitions

Map

Transforms raw input into intermediate key-value pairs.

Reduce

Combines same keys to produce final result.

Why Practicals Less Important in Some Colleges

Your note says theory important.

Because industry now uses:

- Spark (faster)
- Hive
- Databricks

But MapReduce theory is foundation.

If you understand MapReduce, Spark becomes easy.

Interview Questions

Why MapReduce?

To process massive distributed data in parallel.

Why key-value pairs?

Easy grouping and aggregation.

What is Shuffle?

Grouping same keys from all mappers.

Why code goes to data?

Data locality.

Super Easy Memory Trick

MAP = Divide Work

REDUCE = Combine Work

Final Example in 10 Seconds

Input:

cat dog cat

Mapper:

(cat,1)

(dog,1)

(cat,1)

Reducer:

cat = 2

dog = 1

How MapReduce Works Internally (Simple & Neat)

MapReduce processes big files inside Hadoop using **many machines together**.

It works in stages.

Big Picture Flow

Input File in HDFS

↓

Split into blocks

↓

Mapper Tasks

↓

Shuffle + Sort

↓

Reducer Tasks

↓

Final Output in HDFS

Step 1: File Stored in HDFS

Suppose file size = **1 GB**

HDFS breaks file into blocks (example 128 MB each).

So:

\frac{1024}{128}=8

So file becomes **8 blocks**.

Stored across many DataNodes.

Step 2: Job Submitted

When you run MapReduce program:

hadoop jar job.jar

Hadoop starts a job.

It checks:

- where blocks are stored
- available machines
- resources

Step 3: Input Splits Created

Logical splits are created.

Usually:

- 1 block = 1 split
- 1 split = 1 mapper

So 8 blocks → 8 mappers.

Step 4: Mapper Runs Near Data

Important concept:

Data Locality

Hadoop sends mapper code to machine where block exists.

Because:

- code is small
- data is big

This saves network time.

Example

Block1 contains:

hello hello cat

Mapper output:

(hello,1)

(hello,1)

(cat,1)

Each mapper does same for its block.

Step 5: Intermediate Output Stored

Mapper output is temporary.

Stored on local disk.

Not final output.

Step 6: Shuffle Phase

Now Hadoop moves mapper outputs to reducers.

Same keys go to same reducer.

Example:

hello -> from mapper1, mapper3, mapper7

cat -> from mapper1, mapper2

This network movement is called:

Shuffle

Step 7: Sort Phase

Before reducer starts:

Keys are sorted/grouped.

Example:

cat -> 1,1

hello -> 1,1,1,1

zebra -> 1

Step 8: Reducer Runs

Reducer gets:

hello -> [1,1,1,1]

Reducer adds:

hello = 4

Final output:

cat = 2

hello = 4

Step 9: Final Output Saved in HDFS

Reducer writes result back to HDFS.

Files like:

part-00000

part-00001

Internal Components (Old Hadoop)

Component	Work
NameNode	Knows where blocks are stored
DataNode	Stores file blocks
JobTracker	Manages MapReduce job
TaskTracker	Runs tasks

(In newer Hadoop: YARN uses ResourceManager / NodeManager)

What If Machine Fails?

If mapper machine crashes:

- Hadoop detects failure

- reruns task on another machine
- uses replica block copy

This gives fault tolerance.

Why It Is Fast

Instead of 1 machine reading full 1 GB:

8 mappers process 8 parts together.

Parallel processing = faster.

Example Full Internal Flow

1 GB File

↓

8 Blocks

↓

8 Mappers

↓

(word,1)

↓

Shuffle network transfer

↓

Reducers

↓

(word,total)

↓

Output in HDFS

Easy Memory Trick

Mapper = Produce small results

Shuffle = Group same keys

Reducer = Final combine

Honest Industry Truth

Modern companies often use **Spark**, but Spark still uses same concepts:

- partition data
- parallel tasks
- combine results

So understanding MapReduce gives strong foundation.

One-Line Final Answer

Internally MapReduce splits file into blocks, runs mappers in parallel near data, shuffles same keys to reducers, reducers combine values, and stores final result.